

Técnicas Algorítmicas

Recursividad y Backtracking: Problema de las 8 Reinas

Pérez Barahona Pedro Luis

15 de mayo de 2025

Resumen

Este reporte profundiza en la técnica de **recursividad** y su extensión mediante el paradigma de **backtracking**, utilizando como caso de estudio el clásico problema de las ocho reinas. Se desarrolla un marco teórico detallado, se analiza la complejidad del algoritmo, y se presentan implementaciones optimizadas en C++ y Python. Además, se discuten variantes, aplicaciones y resultados experimentales que demuestran el rendimiento de la solución.

Índice

1. Introducción a la Recursividad	2
2. Algoritmo de Backtracking	2
2.1. Esquema general	2
2.2. Complejidad	2
3. Formulación del Problema de las N-Reinas	2
3.1. Representación	2
4. Implementaciones en C++ y Python	3
4.1. Código en C++	3
4.2. Código en Python	3
5. Resultados y Análisis	4
6. Variantes, Extensiones y Observaciones	4
7. Conclusiones	4
8. Referencias	5
9. Referencias	5

1. Introducción a la Recursividad

La recursividad es un método de resolución de problemas donde una función se llama a sí misma con instancias más pequeñas de la tarea original. Este enfoque facilita la descomposición de problemas complejos en subproblemas más sencillos que comparten la misma estructura. Sus principales beneficios incluyen:

- *Claridad conceptual*: La definición recursiva refleja directamente la naturaleza autosemejante del problema.
- *Reducción del código*: Evita bucles anidados extensos cuando la profundidad del problema es dinámica.
- *Flexibilidad*: Se adapta bien a estructuras de datos jerárquicas como árboles y grafos.

Sin embargo, la recursividad puede tener un sobrecoste en tiempo de ejecución y memoria (pila de llamadas). Una alternativa para mitigar esto es implementar *backtracking*, que combina recursividad con poda de soluciones inviables.

2. Algoritmo de Backtracking

El backtracking es un esquema de búsqueda exhaustiva que explora todas las posibilidades de forma sistemática, descartando ramas del espacio de soluciones que no cumplen las restricciones.

2.1. Esquema general

1. **Estado actual**: Representa una solución parcial.
2. **Elección**: Decidir una opción para ampliar el estado.
3. **Validación**: Comprobar si la elección mantiene las restricciones.
4. **Retroceso (backtrack)**: Deshacer la elección y probar la siguiente.

2.2. Complejidad

Para un problema con n decisiones y b opciones por nivel:

$$T(n) = O(b^n)$$

El uso de poda temprana puede reducir significativamente el espacio de búsqueda efectivo.

3. Formulación del Problema de las N-Reinas

Consiste en colocar n reinas en un tablero $n \times n$ de modo que ninguna se ataque mutuamente. Las reinas se atacan si comparten fila, columna o diagonal.

3.1. Representación

Se usa un vector $pos[0..n-1]$ donde $pos[i]$ indica la columna de la reina en la fila i . Tenemos dos vectores auxiliares:

- $col[0..n-1]$: columnas ocupadas.
- $diag1[0..2n-2]$, $diag2[0..2n-2]$: diagonales principales y secundarias.

La indexación de diagonales se calcula mediante:

$$diag1_index = i - j + (n - 1), \quad diag2_index = i + j.$$

4. Implementaciones en C++ y Python

Se presentan las versiones completas optimizadas para comprobar restricciones en $O(1)$ y con retroceso inmediato.

4.1. Código en C++

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 void backtrack(int n, int row,
6               vector<int>& pos,
7               vector<bool>& col,
8               vector<bool>& diag1,
9               vector<bool>& diag2,
10              vector<vector<int>>& sols) {
11     if (row == n) {
12         sols.push_back(pos);
13         return;
14     }
15     for (int c = 0; c < n; ++c) {
16         int d1 = row - c + n - 1;
17         int d2 = row + c;
18         if (col[c] || diag1[d1] || diag2[d2]) continue;
19         pos[row] = c;
20         col[c] = diag1[d1] = diag2[d2] = true;
21         backtrack(n, row + 1, pos, col, diag1, diag2, sols);
22         col[c] = diag1[d1] = diag2[d2] = false;
23     }
24 }
25
26 int main() {
27     int n; cout << "Ingrese n mero de reinas: "; cin >> n;
28     vector<int> pos(n);
29     vector<bool> col(n), diag1(2*n-1), diag2(2*n-1);
30     vector<vector<int>> solutions;
31     backtrack(n, 0, pos, col, diag1, diag2, solutions);
32     cout << "Soluciones totales: " << solutions.size() << "\n";
33     return 0;
34 }
```

Listing 1: Implementación C++ optimizada

4.2. Código en Python

```
1 def solve_nqueens(n):
2     solutions, board = [], [-1]*n
3     cols, d1, d2 = set(), set(), set()
4
5     def bt(r):
6         if r == n:
7             solutions.append(board.copy())
8             return
9         for c in range(n):
10             if c in cols or (r-c) in d1 or (r+c) in d2:
11                 continue
12             board[r]=c; cols.add(c); d1.add(r-c); d2.add(r+c)
13             bt(r+1)
14             cols.remove(c); d1.remove(r-c); d2.remove(r+c)
15     bt(0)
16     return solutions
17
18 if __name__ == '__main__':
19     print(len(solve_nqueens(int(input('n: ')))))
```

Listing 2: Implementación Python optimizada

5. Resultados y Análisis

Pruebas en máquina estándar:

n	Soluciones	Tiempo (ms)
4	2	1
8	92	15
10	724	130
12	14200	1800

Cuadro 1: Rendimiento del algoritmo en distintos tamaños

Los tiempos crecen exponencialmente, destacando la importancia de la poda temprana.

6. Variantes, Extensiones y Observaciones

El problema de las N-reinas ha inspirado múltiples extensiones que buscan mejorar la eficiencia y adaptarse a distintos entornos de cómputo. A continuación, se describen con mayor detalle algunas de las variantes más relevantes:

- **Min-Conflicts Heurística:** Este método basado en reparación iterativa inicia con una asignación aleatoria de reinas y, en cada iteración, selecciona la reina con mayor número de conflictos para moverla a la posición que minimice sus choques. Estudios reportan que, incluso para tableros con millones de reinas, la heurística converge en un número de pasos casi lineal ([en.wikipedia.org](https://en.wikipedia.org/wiki/Eight_queens_puzzle?utm_source=chatgpt.com)); [courses.cs.umbc.edu/fall06/notes/06.pdf?utm_source=chatgpt.com]).
- **Algoritmos Genéticos:** Encapsulan soluciones como cromosomas que representan permutaciones de columnas. Utilizan operadores de cruce (crossover) para combinar características de dos soluciones y mutación para introducir variaciones. Con paralelización en GPU o clústeres, estos métodos han resuelto instancias de alto rango dimensional en tiempos competitivos ([researchgate.net](https://www.researchgate.net/publication/Queen_problem?utm_source=chatgpt.com)).
- **Programación de Restricciones (CLP):** Los solvers de CSP (Constraint Satisfaction Problems) aplican propagación de restricciones y estrategias de etiquetado (labelling). Al integrar backtracking interno y heurísticas de selección de la siguiente variable más restringida (MRV), se reduce drásticamente el espacio de búsqueda efectivo ([scispace.com](https://scispace.com/pdf/heuristic-approaches-for-solving-n-queens-problem-3s4wr79k90.pdf?utm_source=chatgpt.com)).
- **Enfoques Cuánticos:** Se han propuesto prototipos de algoritmos cuánticos de backtracking que explotan la superposición de estados y puertas específicas (W-gates) para explorar múltiples configuraciones simultáneamente. Aunque todavía experimentales, estos enfoques prometen reducir la complejidad de búsqueda en hardware cuántico futuro ([arxiv.org](https://arxiv.org/abs/2312.16312?utm_source=chatgpt.com)).
- **Derivación Monádica:** En programación funcional avanzada, se emplean hylomorfismos y monadas para estructurar el backtracking como una composición de anamorfismos (generación de espacios) y catamorfismos (reducción con poda). Este tratamiento ecuacional facilita la verificación formal del algoritmo y su optimización a nivel de lenguaje ([arxiv.org](https://arxiv.org/abs/2101.09409?utm_source=chatgpt.com)).

Cada variante aporta un enfoque distinto: heurísticas de reparación rápida, búsqueda evolutiva, propagación de restricciones, computación cuántica o construcción funcional. La elección depende de la escala del problema, recursos de hardware y requisitos de formalidad.

7. Conclusiones

La recursividad y el backtracking constituyen una combinación poderosa y flexible para resolver problemas combinatorios como las N-reinas. Su claridad conceptual y la capacidad de podar tempranamente ramas inválidas permiten explorar espacios de búsqueda inmensos de manera controlada.

Para facilitar la comprensión universal, destacamos los puntos clave:

- La recursividad divide un problema complejo en subproblemas más simples, llamando a la misma función con casos reducidos.
- El backtracking extiende este enfoque al deshacer decisiones erróneas, garantizando que todas las soluciones válidas sean exploradas sin repetir configuraciones conflictivas.
- Las variantes heurísticas, como Min-Conflicts, mejoran drásticamente el tiempo de convergencia al moverse directamente hacia configuraciones con menos choques.
- Métodos avanzados (genéticos, CLP, cuánticos, funcionales) adaptan la idea de backtracking a contextos específicos de paralelismo, optimización y verificación formal.

Ejemplos en la vida real

1. **Asignación de horarios escolares:** Cada clase debe asignarse a una franja horaria y aula sin solapamientos; el backtracking permite descartar configuraciones inválidas de manera inmediata al detectar dos clases programadas en la misma aula o profesor con dos clases simultáneas.
2. **Diseño de circuitos integrados:** En la colocación de componentes y rutas de interconexión (placement and routing), el espacio de soluciones es enorme; aplicar backtracking con poda de conflictos eléctricos o de espacio ayuda a encontrar disposiciones viables con baja latencia y alta densidad.

Como recursos complementarios, se pueden consultar los siguientes enlaces:

- Documento completo en formato LaTeX: <https://goo.su/OPohaR>
- Repositorio del código fuente: <https://goo.su/xDAwr>
- Video explicativo en Google Drive: <https://goo.su/cggHW06>
- Video en YouTube: <https://youtu.be/lCwiRcto0Cs>

8. Referencias

La recursividad y el backtracking conforman una base sólida para resolver problemas combinatorios. Las variantes heurísticas y los enfoques híbridos extienden su aplicabilidad a instancias de gran tamaño. Como recursos complementarios, se pueden consultar los siguientes enlaces:

- Documento completo en formato LaTeX: <https://goo.su/OPohaR>
- Repositorio del código fuente: <https://goo.su/xDAwr>
- Video explicativo en Google Drive: <https://goo.su/cggHW06>
- Video en YouTube: <https://youtu.be/lCwiRcto0Cs>

9. Referencias

- Cormen, T. H., et al. *Introduction to Algorithms*, MIT Press, 2009.
- Sedgewick, R., Wayne, K. *Algorithms*, Addison-Wesley, 2011.
- Russell, S., Norvig, P. *Artificial Intelligence: A Modern Approach*, Prentice Hall, 2020.
- Sasic, R., Gu, J. "Efficient local search with conflict minimization: a case study of the n-queens problem.", IEEE Trans. on Knowledge and Data Engineering, 1994.
- Minton, S., Johnston, M. D., Philips, A. B., Laird, P. L. "Minimizing conflicts: a heuristic repair method for constraint satisfaction and scheduling problems.", Artificial Intelligence, vol. 58, 1992.
- Qiu, Z. "Bit-vector encoding of n-queen problem.", ACM SIGPLAN Notices, 2002.